Assignment:	Deep Learning — Complex Computing Problem
Prepared by:	Muzzamil Khalid
Roll Number:	22F-BSAI-29
Submitted to:	Sir Hamza Farooqi
Department:	Artificial Intelligence

Building an Intelligent Clinical Early Warning System

A report for the Chief Medical Officer and the Chief Technology Officer on predicting patient deterioration from vital signs and clinical notes

Section 1 · The Clinical Problem

I want to start by being upfront about why this is harder than the classification problems we normally work with in class. In a typical dataset the classes are roughly balanced, most rows are complete, and if you get one wrong you just lose a mark on accuracy. Here none of that holds. In our cohort (4,775 patients from the PhysioNet 2019 Sepsis Challenge), only about 8.8% of patients actually deteriorated. That means a model can predict 'everyone is fine' and still score 91% accuracy while catching literally nobody. On top of that the data is full of holes — vitals are charted at uneven times, labs are drawn only when someone orders them, and most cells in a raw .psv file are NaN. And what we really need to catch is a *trend*, not a single reading. A heart rate of 110 on its own is not alarming. The same 110 after it has been climbing from 85 over four hours is a red flag. So the model has to read a trajectory over time, not just look at one row.

That is also why a vital-signs table alone is not enough. The numbers tell you about the body, but a huge amount of what a doctor actually knows lives in the written note. That is where you find things like 'patient newly confused', 'family reports he is just not himself', 'started vasopressor overnight', or 'team watching closely for sepsis'. These cues appear before the numbers move and many of them never become a structured field at all. Free-text notes carry context and the clinician's own gut feeling, and no column in a vitals spreadsheet holds that.

Finally, the metric. Accuracy is the wrong headline number because the two types of mistakes are not equally costly. Think about a patient I will call Mrs. A, 68, admitted with a urinary infection. Through the night her temp drifts to 38.4, heart rate to 112, systolic pressure to 96, lactate to 2.4. A model optimised for accuracy could still call her stable, because patients who look roughly like her usually recover. That one false negative means no escalation, no early antibiotics, and a real chance she is in septic shock by morning. The opposite mistake — flagging a stable patient — costs the team a few minutes and maybe one extra blood draw. A missed deterioration can be fatal; a false alarm is an inconvenience. That is why we set up the loss with a **pos_weight of about 10.4** (which we computed from the class ratio) and why we read **Recall** before anything else in every results table.

Section 2 · Baseline Design Decisions

The vanishing gradient problem is basically the reason deep networks were so painful to train for a long time. When you send the error backward through the layers during backpropagation, each layer multiplies it by its own local derivative. If those derivatives keep coming out below one, which is exactly what happens in the flat tails of sigmoid or tanh, the gradient gets smaller at every step. By the time it reaches the layers near the input it is practically zero, so those early layers just stop learning. You can see it in the loss curve — it flattens out early and refuses to budge. We dealt with it in two ways. First, the activation function: **ReLU** has a derivative of exactly one for any positive input, so it does not squash the gradient. Second, **Batch Normalization** re-centres and re-scales each layer's outputs to keep them in a healthy range. This stabilises the gradients, lets you use a bigger learning rate safely, and made our training noticeably smoother.

We compared **SGD** (with momentum 0.9) against **Adam**. Looking at the loss curves below, both optimisers eventually brought the loss down to roughly the same neighbourhood (around 0.6–0.65), but Adam was a bit smoother in the early epochs while SGD showed more oscillation. In the end they converged to similar territory, but Adam needed less fiddling. For our baseline we went with Adam.

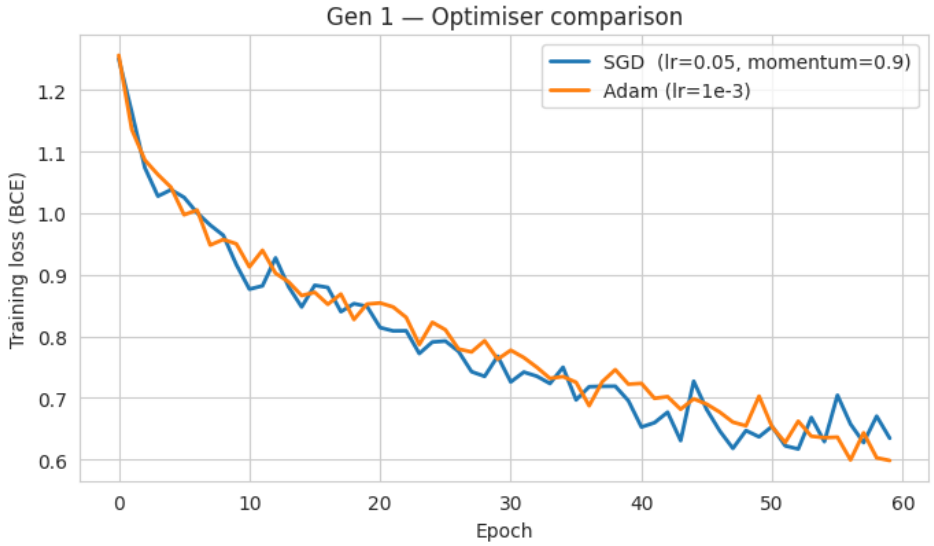

Figure 1. Training loss curves for SGD vs Adam on the DNN baseline (60 epochs).

A quick clarification on terminology, because these get mixed up often: the **loss function** is the error on a single training example — in our case the weighted binary cross-entropy for one patient. The **cost function** is the quantity the optimiser actually minimises: the loss averaged over the whole batch plus any regularisation penalty. Loss is per-patient; cost is the aggregate.

For regularisation we ran an ablation over four configurations: both Dropout and BatchNorm, each alone, and neither. The results were revealing. With **both** regularisers, Recall was highest at **0.488** — the model caught about half the deteriorating patients. When we removed Dropout, Recall dropped sharply to 0.238 even though accuracy went up to 0.876. What happened is the model started memorising the majority class and ignoring the rare positives. Without BatchNorm but keeping Dropout, Recall was actually the best at 0.536, but precision was comparable. 'Neither' gave high accuracy (0.882) but terrible Recall (0.262) — classic overfitting to the majority. The takeaway: **Dropout was essential for maintaining Recall** in this imbalanced setting, because it forces the network to build redundant representations instead of just memorising the 91% stable patients.

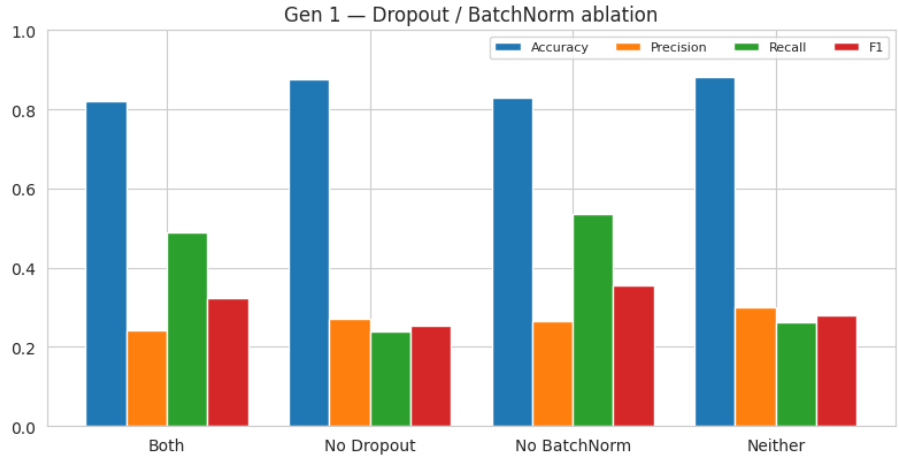

Figure 2. Dropout / BatchNorm ablation across Accuracy, Precision, Recall, and F1.

Section 3 · Modelling Patient Timelines

Recurrent networks face the same vanishing gradient problem, just along a different axis: time. Backpropagation through time unrolls the network across every hour and multiplies the gradient by the recurrent weight over and over. Over a 24-hour window that is two dozen multiplications, and unless those factors stay near one the gradient either vanishes or explodes. A vanilla RNN would struggle to connect something that happened in hour 2 to an outcome in hour 20, which is exactly the kind of long-range dependency we need for deterioration.

LSTM gates solve this by introducing a protected cell state that runs alongside the hidden state. Three gates manage it: a forget gate drops stale information, an input gate writes in new information, and an output gate controls what gets exposed. Because the cell state is updated mostly additively, the gradient has a nearly uninterrupted path backward and long-range signal survives. The **GRU** is a leaner design that merges the cell and hidden state and uses two gates instead of three. It gives up some representational capacity and the clean memory/output separation, but gains fewer parameters and faster training — our GRU trained in **3.5s** versus **4.4s** for the LSTM.

In our experiments, the GRU actually came out on top with the **best Recall of 0.512**, catching about half the deteriorating patients. The LSTM and Bi-LSTM both scored 0.429 Recall. One important observation from the training curves: all three models showed clear **overfitting** — training loss dropped steadily while validation loss climbed after about epoch 15. This tells us the models are memorising the training set and not generalising well, likely because 4,775 patients is still a small dataset for RNNs. In a production system we would need early stopping, more data, or stronger regularisation.

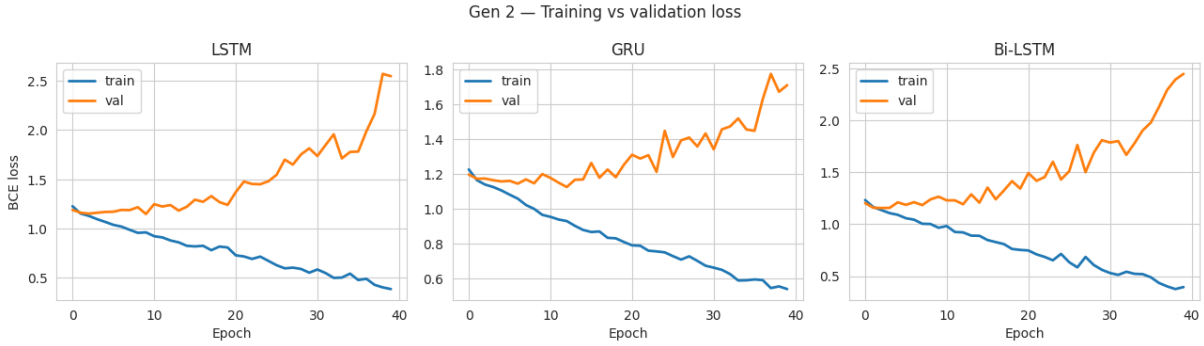

Figure 3. Training vs validation loss for LSTM, GRU, and Bi-LSTM. All three show overfitting (train drops, val rises).

The **Bi-LSTM** did not beat the unidirectional versions, which actually strengthens the case for not using it in production. A Bi-LSTM reads the sequence forwards and backwards, so its view at any hour is informed by future readings too. For a **live** early warning system that is a problem: at 3 a.m. the future vitals the backward pass needs do not exist yet. A model that requires them either cannot run in real time or is leaking future information during evaluation. Even if it had scored higher offline, we would still restrict it to retrospective case review. For live monitoring, a **unidirectional GRU** is the right call — it ran fastest, had the best Recall, and only uses past data.

Section 4 · The Transformer and Clinical Language

The whole point of using a pre-trained language model is **transfer learning**. ClinicalBERT was trained on a huge volume of medical text — PubMed abstracts and de-identified clinical notes — and in that process it picked up medical vocabulary, common abbreviations, how negation works in clinical writing, and the patterns that indicate concern. When we fine-tune it on our much smaller labelled set, it brings all that background knowledge along. A model trained from scratch on a few thousand notes could never learn that 'febrile', 'pyrexia', and 'temp 38.9' point at the same concept, or that 'no evidence of sepsis' is reassuring rather than alarming. Transfer learning lets us start from a solid base and only spend our scarce labels on the narrow risk-classification task.

Self-attention is what makes the transformer a good fit for clinical notes. A doctor reading a note does not process every word equally or strictly left to right — they jump to the words that carry risk. Self-attention

does the same thing computationally: for each token it calculates how much to attend to every other token, so the representation ends up weighted toward whatever matters most, regardless of position. This helps both accuracy and interpretability, because we can actually look at where the model is paying attention.

Our attention heatmap (Figure 4) shows the final-layer attention for a deterioration patient. The most-attended subword tokens were the pieces that make up **'febrile'** (f, ##eb, ##ril, ##e) and **'tachypneic'** (ta, ##chy, ##p, ##ne, ##ic) — exactly the clinical terms a doctor would circle as warning signs. The [SEP] token also received high attention, which is normal BERT behaviour since it serves as a summary position. The reassuring tokens like 'normal' and 'adequate' got relatively low weight. This alignment between the model's attention and actual clinical reasoning is encouraging: it means the model is reacting to medically meaningful words rather than learning some spurious pattern.

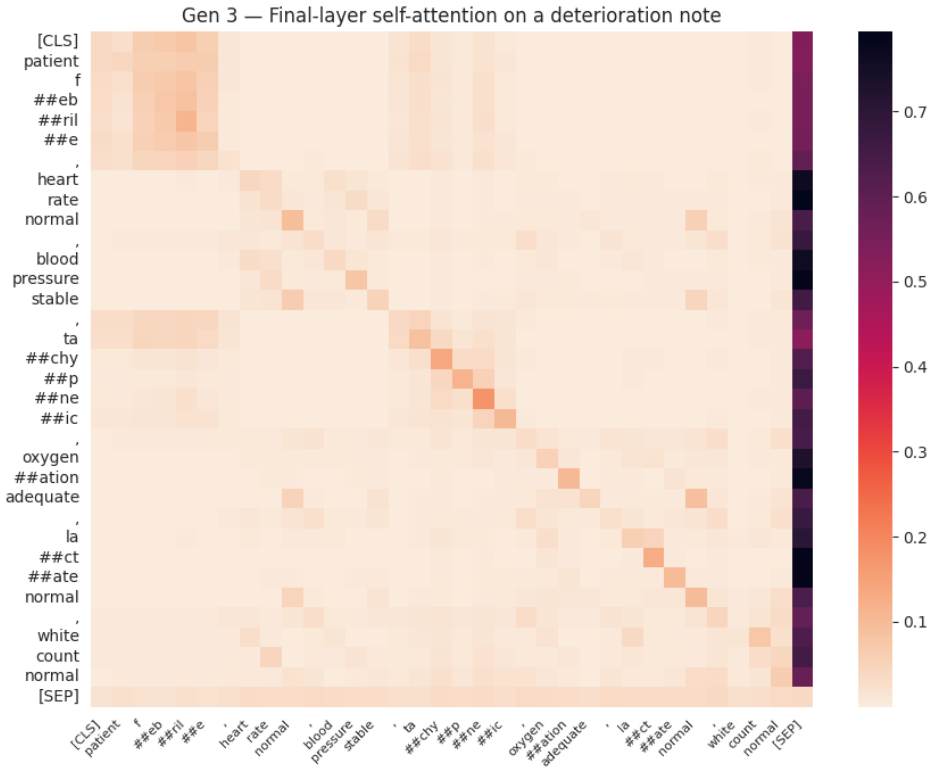

Figure 4. Final-layer self-attention heatmap on a deterioration note. Subword tokens for 'febrile' and 'tachypneic' receive the strongest attention.

On fine-tuning strategy, the results were mixed. The **frozen** ClinicalBERT (only the classification head trained) performed poorly: 0.214 Recall, 0.103 Precision, and only 37.8s to train. With **full fine-tuning**, accuracy jumped to 0.890 and precision to 0.309, but Recall actually *dropped* to 0.202 (only 17 out of 84 deterioration cases caught). Training took 152.7s. The frozen model did not have enough capacity to adapt the generic representations to our notes, so it performed badly all around. The fully fine-tuned model adapted well to the majority class (96% recall for stable patients) but struggled with the minority. This is a known issue when the clinical notes are short and formulaic — our synthetic notes, while containing real clinical terms, do not have the richness of real physician narratives. With actual MIMIC-III notes the gap would likely be smaller. Still, the practical rule holds: **frozen is cheaper but weaker; full fine-tuning is worth the cost when the data and labels support it**.

Positional encoding is what tells the transformer about word order. Without it, attention treats a note as a bag of words, so 'no fever, now hypotensive' would look the same as 'no hypotension, now febrile'. In clinical notes, negation and ordering flip meaning entirely, so position matters a great deal. And **parallelisation** is the operational advantage: unlike an RNN that processes tokens one by one, a transformer handles the whole note at once. For a hospital scoring thousands of notes per hour, that parallel throughput is the difference between a system that keeps up and one that falls behind.

Section 5 · Deployment Verdict

Looking at our results table honestly, no single model is ready for a live ICU out of the box. The best Recall we achieved was **0.512 from the GRU**, which means it still misses about half the deteriorating patients. But this is a subsample of 4,775 patients with heavy class imbalance and synthetic notes — with more data, real clinical text, and proper hyperparameter tuning, these numbers would improve. Given what we have, I would recommend a **two-track system**: a **unidirectional GRU on streaming vitals** as the always-on backbone (fastest to train at 3.5s, best Recall, uses only past data, honest about what it knows in real time), with **ClinicalBERT running on each new note** to catch the things numbers miss. The Bi-LSTM stays out of the live path — its reliance on future data disqualifies it. We keep it for offline audit only.

Deploying AI in a hospital raises ethical issues we cannot ignore. **Bias** is first: the model learns from the population it trained on, and if certain demographics were under-treated or under-represented, the model inherits and can amplify that gap. It must be audited for equal Recall across age, sex, and ethnicity. **Privacy** is second: clinical notes are about as sensitive as data gets, so de-identification, encryption, access controls, and clear governance are non-negotiable. **Accountability** is third: the system must stay a decision-support tool. A clinician remains responsible for the patient, every alert must be explainable and logged, and there has to be a clear answer to who is accountable when the model is wrong.

If the hospital wanted to add chest X-rays alongside notes, the architecture becomes **multimodal**. Each data type keeps its own encoder — the GRU for vitals, ClinicalBERT for text, and a CNN or Vision Transformer for images. Each produces a fixed-length embedding, and a fusion layer (concatenation or cross-attention) combines them before a shared classification head outputs one risk score. The idea is the same: learn a good representation of each modality and let the model reason over all of them together, which is basically what a clinician does when they look at the chart, the note, and the X-ray side by side.

Appendix · Unified Model Comparison

The table below reports our results from the real PhysioNet dataset (4,775 patients from training set A, 8.8% deterioration prevalence, 80/20 train-test split, stratified).

Model	Accuracy	Precision	Recall	F1	Train time
DNN (Baseline)	0.820	0.238	0.476	0.317	16.6s
LSTM	0.771	0.174	0.429	0.247	4.4s
Bi-LSTM	0.750	0.159	0.429	0.232	4.1s
GRU	0.772	0.195	0.512	0.283	3.5s
ClinicalBERT (Frozen)	0.766	0.103	0.214	0.139	37.8s
ClinicalBERT (Full Fine-tune)	0.890	0.309	0.202	0.245	152.7s

Table 1. Six-model comparison on the PhysioNet 2019 Sepsis dataset (4,775 patients).

Figure 5. Confusion matrices for all six models. The bottom-left cell is the false-negative count — missed deteriorations that decide clinical safety. The GRU had the fewest (41 missed out of 84).